

EC2x&EG9x&EG25-G 系列 QuecOpen 日志系统应用指导

LTE Standard 模块系列

版本：EC2x&EG9x&EG25-G 系列 QuecOpen_日志系统应用指导_V1.0

日期：2020-07-07

状态：受控文件

上海移远通信技术股份有限公司始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 51086236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：

<http://www.quectel.com/cn/support/sales.htm>

如需技术支持或反馈我司技术文档中的问题，可随时登陆如下网址：

<http://www.quectel.com/cn/support/technical.htm>

或发送邮件至：support@quectel.com

前言

上海移远通信技术股份有限公司提供该文档内容用以支持其客户的产品设计。客户须按照文档中提供的规范、参数来设计其产品。由于客户操作不当而造成的人身伤害或财产损失，本公司不承担任何责任。在未声明前，上海移远通信技术股份有限公司有权对该文档进行更新。

版权申明

本文档版权属于上海移远通信技术股份有限公司，任何人未经我司允许而复制转载该文档将承担法律责任。

版权所有 ©上海移远通信技术股份有限公司 2020，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2020.

文档历史

修订记录

版本	日期	作者	变更表述
1.0	2020-07-07	匡昌胜/ 孙保	初始版本

目录

文档历史	2
目录	3
表格索引	4
1 引言	5
1.1. 适用模块	5
2 日志系统.....	6
2.1. 日志系统概述.....	6
2.2. 日志的使用	6
2.3. 日志的查看	7
2.4. 日志的配置	8
3 应用程序异常记录	9
3.1. 监听应用程序异常	9
3.2. 获取异常调用栈	10
4 注意事项	11
5 附录 A 术语缩写	12

表格索引

表 1: 适用模块 5

表 2: 日志系统配置项 8

表 3: 术语缩写 12

1 引言

移远通信 LTE Standard EC2x&EG9x&EG25-G 系列模块支持 QuecOpen®方案，本文档主要描述这些模块日志系统的使用、查看和配置，以及相关应用程序异常记录以及注意事项。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
EC2x 系列	EC25 系列
	EC21 系列
	EC20 R2.1
	EC20-CN
EG9x 系列	EG95 系列
	EG91 系列
EG25-G	EG25-G

2 日志系统

2.1. 日志系统概述

EC2x&EG9x&EG25-G 系列 QuecOpen®方案中的日志系统，采用 Android 的 Logcat 方案。Logcat 包含 4 个环形读写缓冲区（MAIN、RADIO、EVENTS 和 SYSTEM）和 6 个日志等级（VERBOSE、DEBUG、INFO、WARN、ERROR 和 FATAL）；4 个环形读写缓冲区的使用情况如下：

MAIN	客户应用进程使用
RADIO	弃用
EVENTS	记录系统事件
SYSTEM	记录系统关键服务进程日志

可使用 Logcat 命令查看日志，并根据 tag 和日志等级进行过滤，具体用法可参考 <https://developer.android.com/studio/command-line/logcat>。

2.2. 日志的使用

日志既可采用 Android 封装的 liblog 系统，也可采用 QuecOpen®方案中封装的 qlsyslog 系统。本章节主要介绍 qlsyslog 的使用。

头文件：qlsyslog/ql_sys_log.h
库：libql_sys_log.so

日志接口定义位于头文件 qlsyslog/ql_sys_log.h 中。日志接口的使用可参考如下的示例代码，并在 Makefile 中，将 -lql_sys_log 添加到链接选项中。

```
#include "qlsyslog/ql_sys_log.h"

#define LOG_TAG "fortest"

int main(int argc, char *argv)
{
    QLOGV(LOG_TAG, "I am QL_SYS_LOG_VERBOSE");
    QLOGD(LOG_TAG, "I am QL_SYS_LOG_DEBUG");
}
```

```
QLOGI(LOG_TAG, "I am QL_SYS_LOG_INFO");
QLOGW(LOG_TAG, "I am QL_SYS_LOG_WARN");
QLOGE(LOG_TAG, "I am QL_SYS_LOG_ERROR");
QLOGF(LOG_TAG, "I am QL_SYS_LOG_FATAL");

return 0;
}
```

2.3. 日志的查看

请使用 Logcat 工具查看日志，Logcat 工具的使用指南参考 <https://developer.android.com/studio/command-line/logcat>。

本章节以第 2.2 章的示例代码为例，简单介绍 Logcat 过滤、查看日志的三种使用方法。

1. 通过 tag 过滤日志：

```
/data # logcat -s fortest
----- beginning of system
----- beginning of main
07-17 12:24:11.472 2039 2039 v fortest : I am QL_SYS_LOG_VERBOSE
07-17 12:24:11.472 2039 2039 d fortest : I am QL_SYS_LOG_DEBUG
07-17 12:24:11.472 2039 2039 i fortest : I am QL_SYS_LOG_INFO
07-17 12:24:11.472 2039 2039 w fortest : I am QL_SYS_LOG_WARN
07-17 12:24:11.472 2039 2039 e fortest : I am QL_SYS_LOG_ERROR
07-17 12:24:11.472 2039 2039 f fortest : I am QL_SYS_LOG_FATAL
```

2. 通过等级过滤日志：

```
/data # logcat -s fortest:w
----- beginning of system
----- beginning of main
07-17 12:24:11.472 2039 2039 w fortest : I am QL_SYS_LOG_WARN
07-17 12:24:11.472 2039 2039 e fortest : I am QL_SYS_LOG_ERROR
07-17 12:24:11.472 2039 2039 f fortest : I am QL_SYS_LOG_FATAL
```

3. 通过缓冲区过滤日志

```
/data # logcat -b main
07-17 12:24:11.472 2039 2039 v fortest : I am QL_SYS_LOG_VERBOSE
07-17 12:24:11.472 2039 2039 d fortest : I am QL_SYS_LOG_DEBUG
07-17 12:24:11.472 2039 2039 i fortest : I am QL_SYS_LOG_INFO
07-17 12:24:11.472 2039 2039 w fortest : I am QL_SYS_LOG_WARN
07-17 12:24:11.472 2039 2039 e fortest : I am QL_SYS_LOG_ERROR
07-17 12:24:11.472 2039 2039 f fortest : I am QL_SYS_LOG_FATAL
```

2.4. 日志的配置

qlsyslog 系统会根据过滤规则将内核日志和应用程序日志保存到 `log_file` 指定的位置中。配置文件位于 `/data/qllog.json`，配置文件为 JSON 格式，主要配置选项如下表。

表 2：日志系统配置项

配置项	选项	描述
<code>log_file</code>	必选	日志文件保存位置。在系统运行中，可能会对日志文件频繁写入，不可将日志保存在关键系统分区。
<code>rotate_file_size</code>	必选	单个日志文件大小限制。单位：KB。
<code>rotate_file_count</code>	必选	最大日志文件个数。
<code>log_format</code>	可选	日志文件输出格式。可选 <code>default</code> 、 <code>csv</code> 。
<code>kernel_priority</code>	可选	内核日志等级。内核日志等级即 <code>printk</code> 的等级，可选 <code>m</code> （突发性事件消息）、 <code>a</code> （示警消息）、 <code>c</code> （严重的软硬件错误消息）、 <code>e</code> （错误信息）、 <code>w</code> （关于问题状况的警告信息）、 <code>n</code> （安全相关报告信息）、 <code>i</code> （信息消息）、 <code>d</code> （调试信息）和 <code>*</code> （所有等级）。
<code>buffer_list.{i}.name</code>	必选	缓冲区名称。可选 <code>main</code> 、 <code>system</code> 、 <code>events</code>
<code>buffer_list.{i}.filter_list.{i}.tag</code>	可选	需要保存日志的 <code>tag</code> 。若未设置，默认使用所有 <code>tag</code> 。
<code>buffer_list.{i}.filter_list.{i}.priority</code>	可选	需要保存日志的等级。可选 <code>v</code> （VERBOSE）、 <code>d</code> （DEBUG）、 <code>i</code> （INFO）、 <code>w</code> （WARN）、 <code>e</code> （ERROR）、 <code>f</code> （FATAL）和 <code>*</code> （所有等级）。

3 应用程序异常记录

3.1. 监听应用程序异常

应用程序可通过监听信号来记录异常相关信息，示例代码如下。在异常处理程序中，函数 `ql_sys_log_signal(qlsyslog/ql_sys_log.h)` 将尽可能多的记录程序的异常信息。

```
#include <signal.h>
#include <stdlib.h>
#include "qlsyslog/ql_sys_log.h"

#define LOG_TAG "fortest"

static void handle_signal (int sig_num, siginfo_t *info, void *ptr)
{
    ql_sys_log_signal(QL_SYS_LOG_ID_MAIN, QL_SYS_LOG_FATAL, LOG_TAG, sig_num, info, ptr);
    exit(-1);
}

int main(int argc, char *argv)
{
    struct sigaction sa = {0};
    sa.sa_sigaction = handle_signal;
    sa.sa_flags = SA_SIGINFO;
    sigaction (SIGTERM, &sa, NULL);
    sigaction (SIGSEGV, &sa, NULL);
    sigaction (SIGABRT, &sa, NULL);
    sigaction (SIGINT, &sa, NULL);
    sigaction (SIGBUS, &sa, NULL);

    QLOGI(LOG_TAG, "bootup");

    /* other code */

    return 0;
}
```

3.2. 获取异常调用栈

应用程序发生异常时，可通过详细的应用程序调用栈定位异常代码位置。*ql_sys_log_signal* 可以回溯函数调用栈，但同时也有一定的局限性。

- 方法

删除应用程序编译选项中的优化选项-O1、-O2、-O3 和-fomit-frame-pointer，并添加编译选项 -fasynchronous-unwind-tables -rdynamic;

- 局限

有些库在编译时添加了优化选项。若库函数发生异常，可能无法回溯详细的调用栈。

4 注意事项

在系统运行过程中，可能会产生大量的日志，以及频繁写入日志文件。若日志文件存放在 flash 中，会缩短 flash 使用寿命（flash 技术参数有最大写入次数限制）。

在研发测试阶段，可将日志保存在 flash 中，以便于调试。但是在后续阶段，建议将日志文件保存到临时文件系统，如下所示：

```
~ # cat /data/qllog.json
{
  /* log output format : log type, time, pid, tid, tag, msg */
  /* log type : k->kernel, e->events, s->system, m->main */
  /* (mandatory) log file path */
  "log_file":"/tmp/qllog/ql_log.txt",
  /* (optional) rotate log every kbytes, default is unlimit */
  "rotate_file_size": 1024,
  /* (optional) max number of rotated logs, default is 4 */
  "rotate_file_count": 2,
  /* (optional) log format, default or csv */
  "log_format":"default",
}
```

图 1：保存日志文件到临时文件系统

5 附录 A 术语缩写

表 3: 术语缩写

术语	英文全称	中文全称
KB	Kilobyte	千字节
LTE	Long Term Evolution	长期演进